MIS 600 · DATABASE MANAGEMENT SYSTEMS

Chapter One

The Database Environment and Development Process

1

Dr. Nawaf Aljohani
W. Frank Barton School of Business, Wichita State University
Fall 2026

Figures from Hoffer, Topi, Venkataraman, & Bala (2026), Modern Database Management, 14th ed., Pearson

Tonight

- Course introduction: syllabus, grading, the automatic drop policy, Draw.io and DB Fiddle
- A quick look back at the Week 1 ERD starter
- Chapter 1, part 1: why data matter, the vocabulary, file processing versus the database approach
- Chapter 1, part 2: how databases get built (SDLC, prototyping, agile, three schemas)
- Chapter 1, part 3: a small project at Pine Valley Furniture Company
- In-class exercise 2

TONIGHT'S GOALS

- define the key terms of the chapter
- name the five limits of file processing and the ten advantages of the database approach
- list the nine components of a database environment
- explain where database work fits in the SDLC
- read a simple E-R diagram and a simple SQL query

Why data matter

Every organization now runs on data. The ones that organize it well gain an edge.

1

A fast-food chain

A camera watches the drive-through line. Long line: the digital menu board shows items that are quick to prepare. Short line: it shows higher-margin items that take longer.

2

Kroger

Two billion transactions a year from 60 million shoppers. Kroger sells insights about changing consumer behavior to the companies whose products it stocks. A second revenue stream from data it already had.

3

Southwest Airlines

Detailed analytical models of corporate customers let sales staff spot problems and opportunities before competitors do.

None of this works unless someone first decides how to collect, organize, store, and protect the data. That is the job this course prepares you for.

What is a database?

An organized collection of logically related data.

- **Organized:** structured on purpose, not a pile of files
- **Collection:** many pieces of data about many things
- **Logically related:** the pieces connect. A student connects to her sections, an order connects to its customer

ANY SIZE

- **Megabytes:** a salesperson's contacts on a laptop
- **Gigabytes:** a departmental system
- **Terabytes:** a corporate decision support database
- **Petabytes:** data warehouses and data lakes

Data are a stored representation of objects and events that have meaning and importance in the user's environment. Structured data (numbers, text, dates in tables) and unstructured data (documents, photos, video, posts) now live in the same databases.

Data versus information

Information is data processed so that the knowledge of the person using it increases.

DATA

Baker, Kenneth D.	324917628
Doyle, Joan E.	476193248
Finkle, Clive R.	548429344
Lewis, John C.	551742186
McFerran, Debra R.	409723145
Sisneros, Michael	392416582

Facts with no context. You might guess these are names and ID numbers, but you cannot use them.

TWO WAYS TO GET INFORMATION

- Add context (labels, structure)
- Summarize (totals, charts)

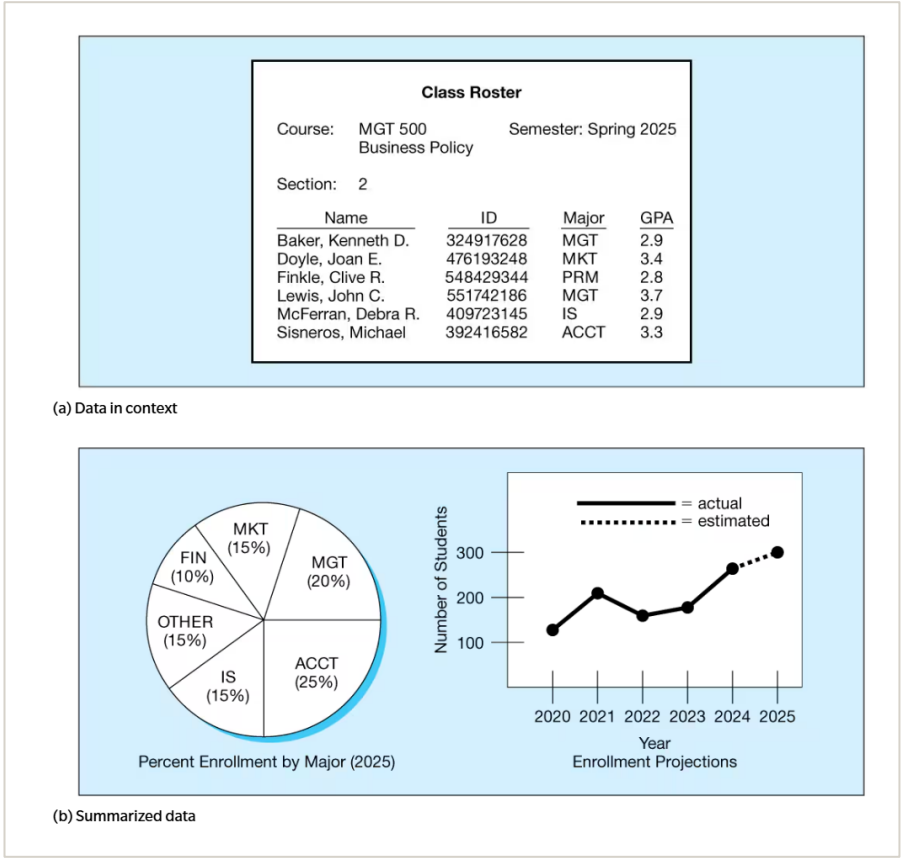


Figure 1-1. Converting data to information (Hoffer et al., 2026)

Metadata: data about data

Data Item			Metadata			
Name	**Type**	**Length**	**Min**	**Max**	**Description**	**Source**
Course	Alphanumeric	30			Course ID and name	Academic Unit
Section	Integer	1	1	9	Section number	Registrar
Semester	Alphanumeric	10			Semester and year	Registrar
Name	Alphanumeric	30			Student name	Student IS
ID	Integer	9			Student ID (SSN)	Student IS
Major	Alphanumeric	4			Student major	Student IS
GPA	Decimal	3	0.0	4.0	Student grade point average	Academic Unit

Table 1-1. Example metadata for the class roster (Hoffer et al., 2026)

- **Properties:** name, definition, data type, length, allowable values (GPA is a decimal between 0.0 and 4.0)
- **Context:** source (system of record), location, ownership, usage
- **Notice:** the table holds no student names and no actual GPAs. Metadata describe the data and stay separate from them
- **Why it matters:** two files both have a column called Date. One means order date, one means ship date. Without metadata you compute nonsense

1

PART 1

Traditional file processing systems

Before databases, every application owned its own files. Understanding what went wrong tells you what a database has to fix.

Pine Valley Furniture before databases

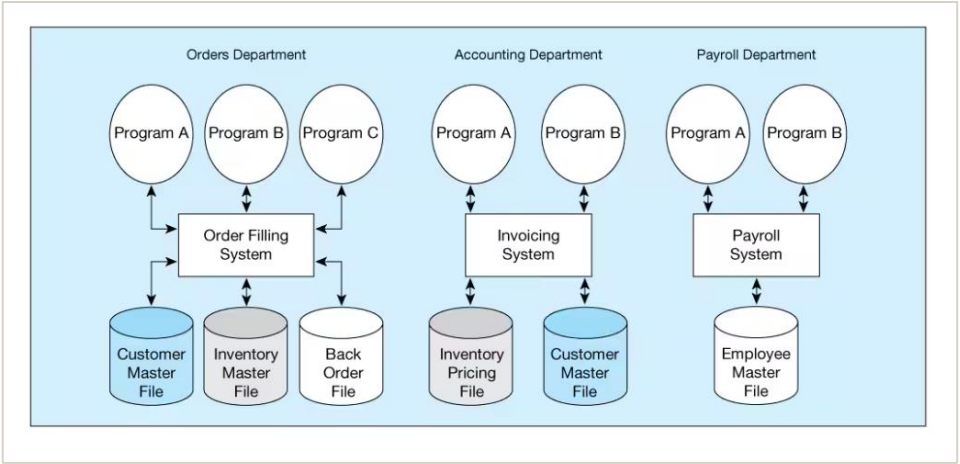


Figure 1-2. Old file processing systems at Pine Valley Furniture Company (Hoffer et al., 2026)

- Each department got its own system and its own files, built one request at a time with no overall plan
- The Customer Master File appears twice: once in Orders, once in Accounting
- Inventory Master (Orders) and Inventory Pricing (Accounting) describe the same products
- A file is a collection of related records. Excel workbooks belong in this picture too

Five disadvantages of file processing

Table 1-2

DISADVANTAGE	WHAT IT MEANS
Program-data dependence	Every program carries its own copy of the file layout. Lengthen the address field from 30 to 40 characters and you must find and change up to five programs.
Duplication of data	Same facts in several files. Copies drift apart: a customer moves, Orders updates its file, Accounting mails the invoice to the old address.
Limited data sharing	Each application owns its files. A report that needs Orders and Payroll data becomes a programming project.
Lengthy development times	Every new application starts from scratch: new file layouts, new access code.
Excessive program maintenance	All of the above add up. As much as 80 percent of the IS budget can go to patching old programs.

Think of each program as a key cut for one lock, and the file layout as the lock. Change the lock and every key must be recut.

Your spreadsheets are file processing systems

- Three student organizations each keep a spreadsheet of members
- A member changes her email and tells one club. The other two are now wrong (duplication)
- Student Government asks for one combined list of active members. Someone spends an afternoon copying, pasting, and removing duplicates (limited sharing, lengthy development)
- The book's warning: informal use of Excel for organizational data is still everywhere, and it has every drawback on the previous slide
- The same drawbacks return in databases if an organization builds many separate ones with no shared metadata. The database approach is a way of managing data, not just a product you buy

2

PART 2

The database approach

Data models, entities and relationships, relational databases, the DBMS, and what you gain and pay for.

Data models: entities, attributes, relationships

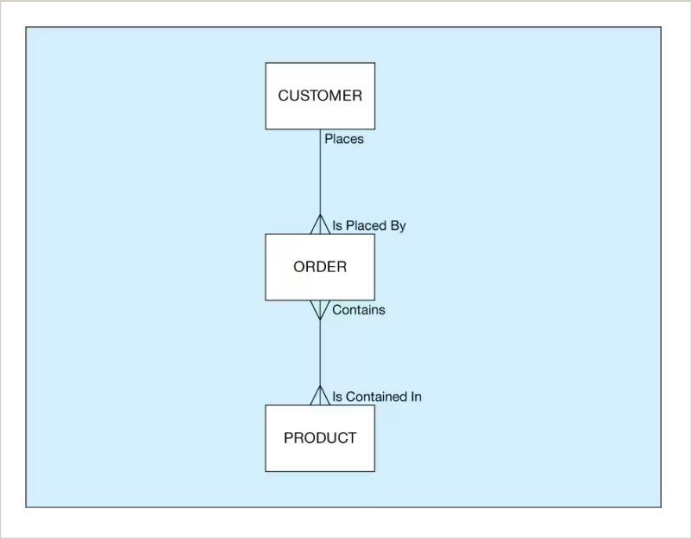


Figure 1-3(a). Enterprise-level segment

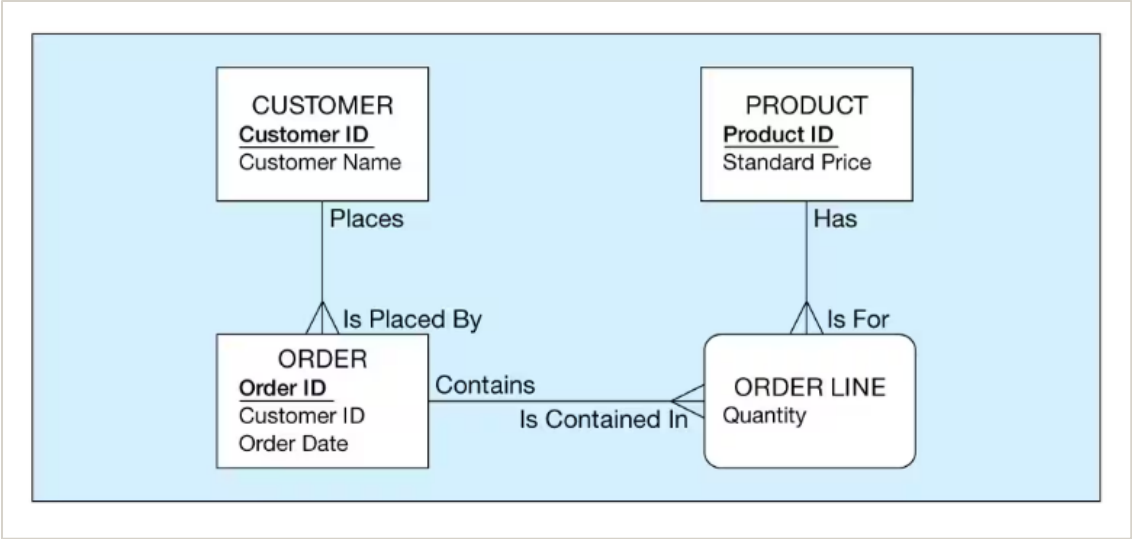
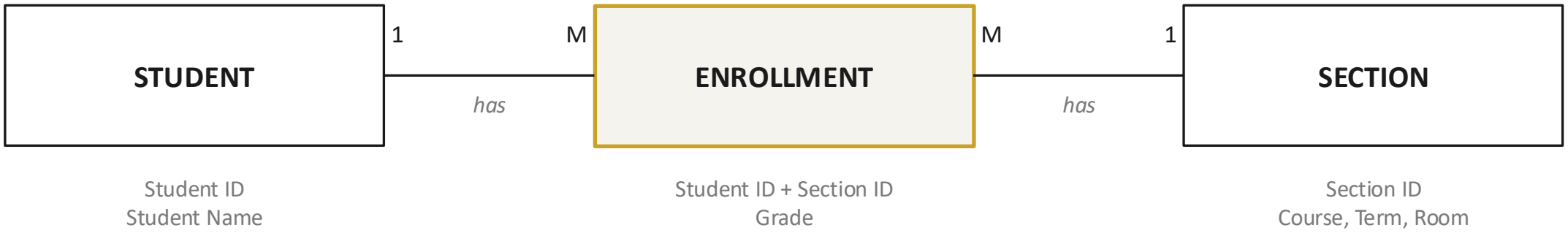


Figure 1-3(b). Project-level segment (Hoffer et al., 2026)

- **Entity:** a noun the business keeps data about (CUSTOMER, ORDER). Attribute: a characteristic (Customer Name). Instance: one customer
- **Relationship:** one-to-many (a customer places many orders) or many-to-many (an order holds many products, a product sits on many orders)
- **Crow's foot:** the three-pronged end means many. Each line has two names, one per direction: Places / Is Placed By
- **Enterprise versus project level:** (b) adds attributes, underlines identifiers, and splits the M:N into two 1:M through ORDER LINE

The same shape on campus

Many-to-many relationships need something in the middle



- A student takes many sections, and a section has many students. That is M:N, exactly like ORDER and PRODUCT
- To record a grade you need something in between. ENROLLMENT holds one student's grade in one section, the way ORDER LINE holds one product's quantity on one order
- If you drew the graduate version of the Week 1 assignment, you already built this

Relational databases and the DBMS

- **Relational database:** tables linked by common fields. Customer ID lives in CUSTOMER and is stored with every ORDER that customer places
- **DBMS:** the software that makes the database approach possible. It creates, updates, stores, and retrieves data, controls access, enforces integrity, manages many users at once, and recovers after failures
- **Products:** Oracle, SQL Server, Db2, MySQL, PostgreSQL, SQLite, Access. NoSQL: MongoDB. In class: DB Fiddle
- **Compare with Figure 1-2:** one CUSTOMER entity instead of two Customer Master Files, one INVENTORY instead of two inventory files

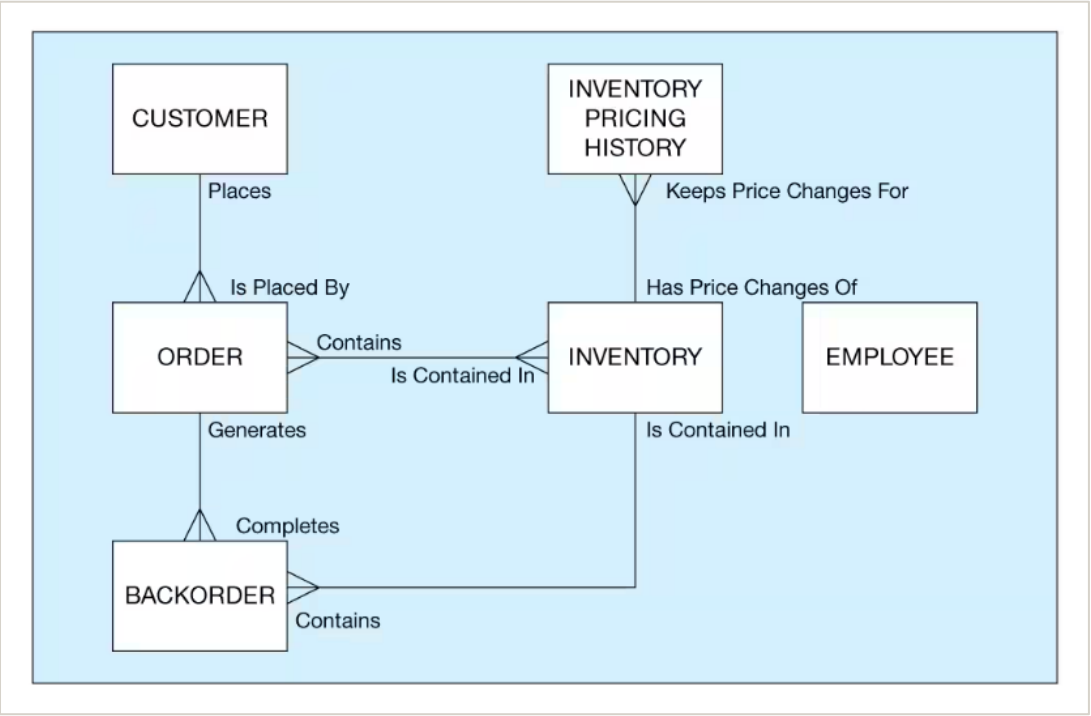


Figure 1-4. Enterprise model for the Figure 1-3 segments (Hoffer et al., 2026)

Ten advantages of the database approach

Table 1-3

- **Program-data independence.** Metadata live in the repository, not in every program
- **Planned data redundancy.** Each fact in one place, with controlled exceptions
- **Improved data consistency.** One copy cannot disagree with itself
- **Improved data sharing.** Each user gets a user view of the part they need
- **Increased productivity of application development.** No file design, reusable tables, built-in tools
- **Enforcement of standards.** Naming, quality, access procedures, set by database administration
- **Improved data quality.** Constraints the DBMS enforces, plus data scrubbing
- **Improved data accessibility and responsiveness.** End users query with SQL
- **Reduced program maintenance.** Change data or programs without always changing both
- **Improved decision support.** Databases built for CRM, finance, supply chain

Caution from the book: many organizations never realize these benefits because of older data models or poor planning. Good software does not rescue a bad plan.

A first taste of SQL

Advantage 8: end users can ask their own questions

```
SELECT *
FROM Product_T
WHERE ProductDescription = 'Computer Desk';
```

- **SELECT *** show every column
- **FROM Product_T** from the Product table
- **WHERE ...** only rows whose description is Computer Desk
- Easy to write, easy to get wrong. Test a query before you trust it

WHAT IT RETURNS (PRODUCT_T, FIGURE 1-17)

ID	DESCRIPTION	FINISH	PRICE	LINE
3	Computer Desk	Natural Ash	375	2
8	Computer Desk	Walnut	250	3

Two of the nine products match, so two rows come back with all their columns. The match is exact: 'computer desk' in lowercase would return nothing in many systems.

Costs and risks of the database approach

Table 1-4. A database is not a silver bullet

COST OR RISK	WHAT IT MEANS
New, specialized personnel	Hire or train database designers and administrators, and keep retraining them as technology changes
Installation and management cost and complexity	A multi-user DBMS is large and expensive, needs trained staff, annual support, and often hardware and network upgrades
Conversion costs	Legacy systems (file based or older DBMS) cost money, time, and commitment to convert
Need for explicit backup and recovery	A shared database must be correct and available all the time, so procedures must exist and be tested
Organizational conflict	Who owns the definition of a customer? Without top management support, stand-alone databases proliferate

Operational versus analytical data management

- **Operational (transactional):** run the business, record every sale and registration. Mostly relational. The core of this course
- **Analytical:** understand the business. Data warehousing (since the late 1980s, mature, mostly relational) and big data (since the early 2010s: volume, variety, velocity)
- **Schema on write:** design the structure before you store anything (a form with fixed boxes)
- **Schema on read:** store first, decide the structure when you use the data (a shoebox of receipts)
- **Our exams follow the figure:** Exam 1 is Chapters 1 to 4, Exam 2 is Chapters 5 and 6, Exam 3 is Chapters 8, 9, 11, and 13

<table>
<tr><th></th><th>Operational</th><th>Analytical</th></tr>
<tr><td>Purpose</td><td>Transactional</td><td>Foundation for analytics</td></tr>
<tr><td>Technology</td><td>Primarily relational</td><td>Relational and non-relational</td></tr>
<tr><td>Modeling</td><td>Conceptual data modeling with (E)ER (Chapters 2 and 3)</td><td rowspan="2">Modern data platforms: integration, warehousing and virtualization (Chapter 10)</td></tr>
<tr><td>Design</td><td>Logical data modeling with the relational model; Normalization (Chapter 4)</td></tr>
<tr><td>Infrastructure</td><td>Physical design of relational databases; Security (Chapter 8)</td><td rowspan="2">Beyond relational: Big Data, NoSQL and cloud databases (Chapter 9)</td></tr>
<tr><td>Access</td><td>SQL (Chapters 5 and 6)
Applications with SQL (Chapter 7)</td></tr>
<tr><td>Data analysis</td><td colspan="2">Analytics: From data to insights (Chapter 11)</td></tr>
<tr><td>Data quality, administration and governance</td><td colspan="2">Life cycle (Chapter 1)
Data quality, administration and governance (Chapter 12)</td></tr>
</table>

Figure 1-5. Integrated data management framework (Hoffer et al., 2026)

Nine components of the database environment

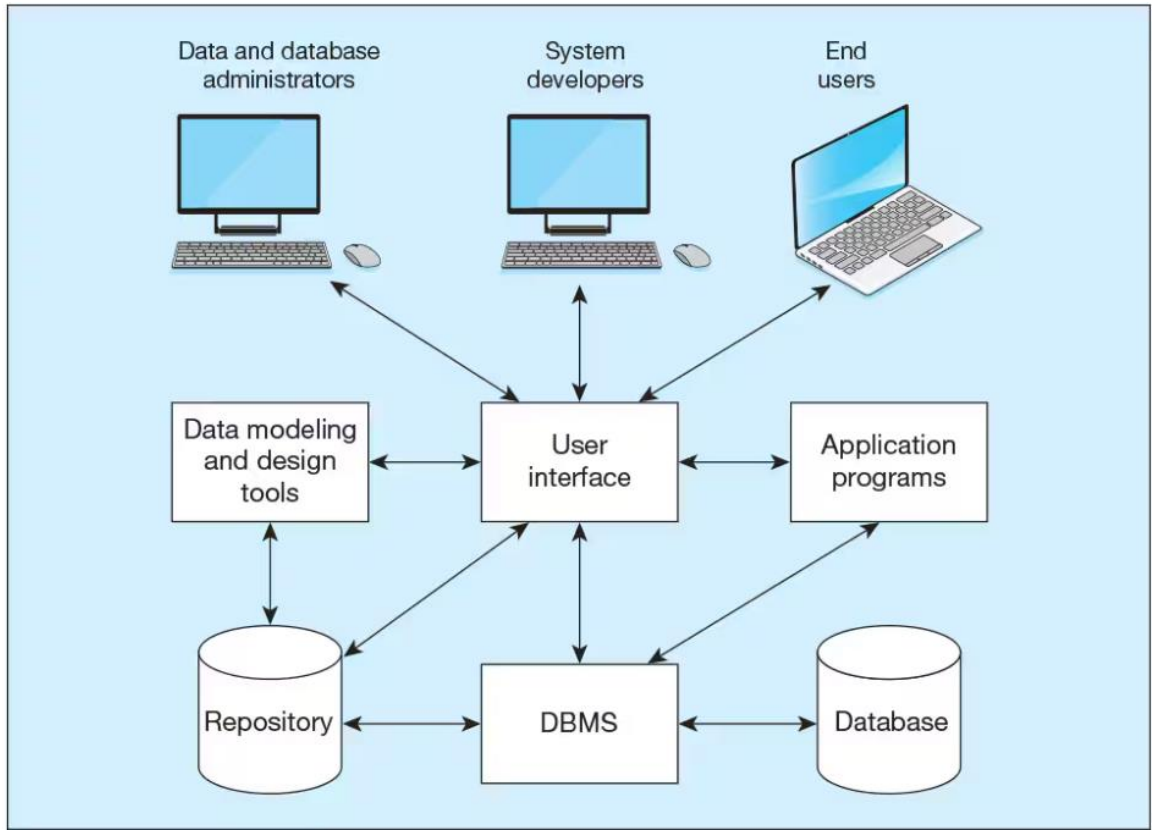


Figure 1-6. Components of the database environment (Hoffer et al., 2026)

- **1 Data modeling and design tools** (Draw.io for us)
- **2 Repository** holds definitions of data (metadata)
- **3 DBMS** (PostgreSQL, MySQL, SQL Server, Oracle)
- **4 Database** holds occurrences of data
- **5 Application programs**
- **6 User interface** (DB Fiddle's editor, a web form)
- **7 Data and database administrators**
- **8 System developers**
- **9 End users**
- Every path to the data goes through the DBMS

3

PART 3

How databases get built

Enterprise data modeling, the SDLC, prototyping and agile, and the three-schema architecture.

Enterprise data modeling

- **Purpose:** an overall picture of the organization's data, not the design of one database
- **Contents:** high-level entities and relationships (Figure 1-3(a)), business descriptions, and business rules
- **The matrix:** rows are business functions, columns are entities, X means the function uses the entity. Order Fulfillment uses all nine
- **Two kinds of projects:** top-down from planning (strategic goals) and bottom-up from users who need information. Most are bottom-up

Business Functions \ Data Entity Types	Customer	Product	Raw Material	Order	Work Center	Work Order	Invoice	Equipment	Employee
Business Planning	X	X						X	X
Product Development		X	X		X			X	
Materials Management		X	X	X	X	X		X	
Order Fulfillment	X	X	X	X	X	X	X	X	X
Order Shipment	X	X		X	X		X		X
Sales Summarization	X	X		X			X		X
Production Operations		X	X	X	X	X		X	X
Finance and Accounting	X	X	X	X	X		X	X	X
X = data entity is used within business function									

Figure 1-7. Business function-to-data entity matrix (Hoffer et al., 2026)

The systems development life cycle

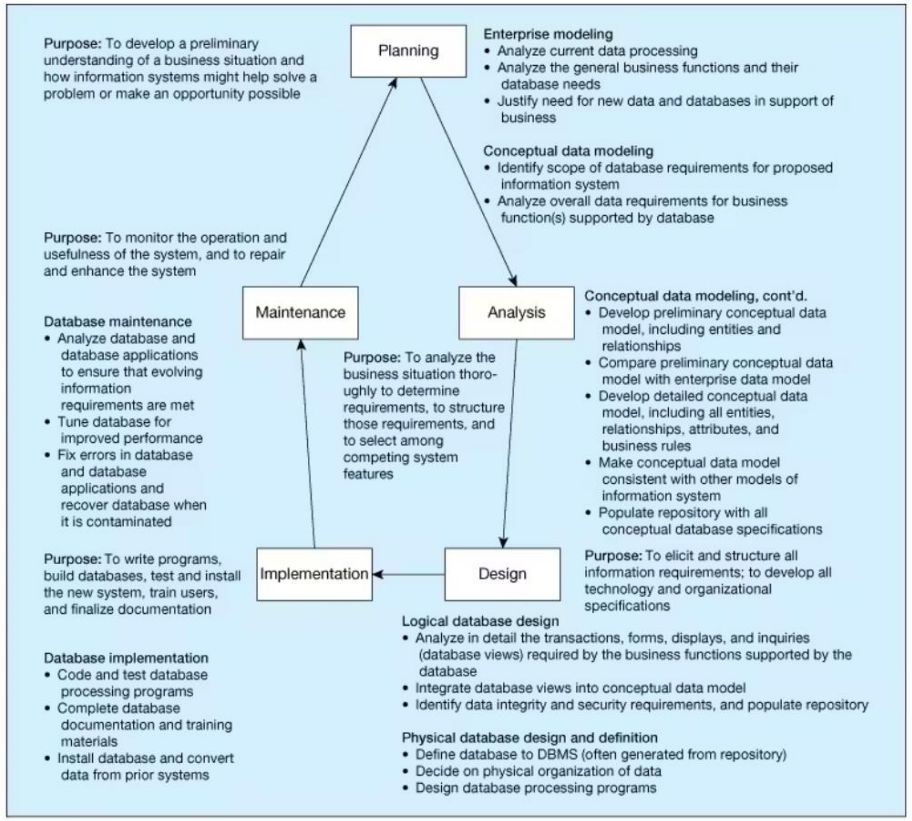


Figure 1-8. Database development activities during the SDLC (Hoffer et al., 2026)

- **Planning:** enterprise modeling, conceptual data modeling begins (scope)
- **Analysis:** detailed conceptual data model with every entity, attribute, relationship, and rule
- **Design:** logical database design (tables, keys, normalization), then physical design (storage, indexes)
- **Implementation:** code, test, install, train, load data, go live
- **Maintenance:** the longest phase. Every change is a small development cycle
- Drawn as a circle on purpose: phases overlap, run in parallel, and backtrack

What each phase produces

SDLC PHASE	DATABASE ACTIVITY	OUTPUT
Planning	Enterprise modeling. Conceptual data modeling: scope of data for this project	Enterprise data model, project scope
Analysis	Conceptual data modeling continued: all entities, attributes, relationships, integrity rules	Conceptual schema
Design	Logical database design: conceptual schema expressed in the chosen technology (relational tables, keys, constraints), user views integrated, normalization	Logical schema
Design	Physical database design: how a specific DBMS stores the data, file organization, indexes	Physical schema (one per logical schema)
Implementation	Write and test programs and scripts, document, train, load data from old systems, go into production	Working database
Maintenance	Add, change, remove structures as the business changes, fix design errors, tune, rebuild after failures	An evolving database

Prototyping: build it, use it, revise it

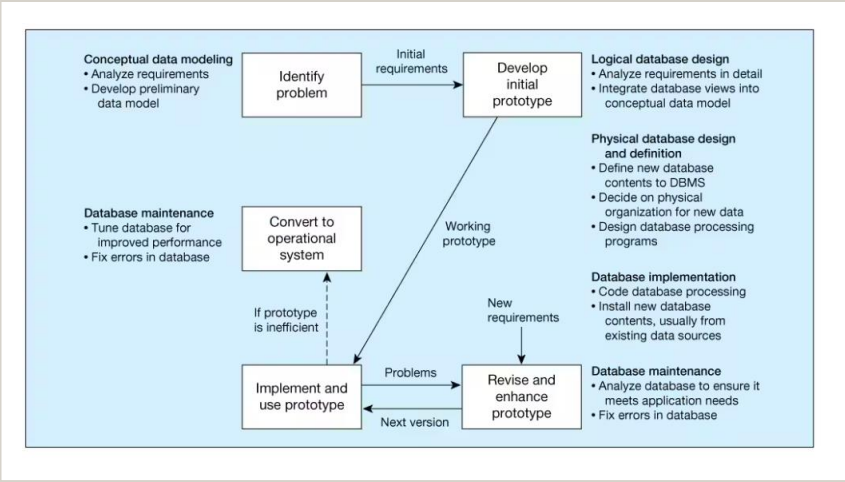


Figure 1-9. The prototyping methodology and database development process (Hoffer et al., 2026)

- A rapid application development method: convert requirements into a working system quickly, then revise it with the users until they are satisfied
- Works best when the database structures already exist and the system mostly retrieves data
- Early versions have minimal security and integrity controls, documentation comes late, users learn by doing
- If the accepted prototype is too inefficient, convert it into an operational system. Speed is not an excuse for bad design

Agile software development

The 2001 Manifesto for Agile Software Development (Beck et al., 2001)

Individuals and interactions

over processes and tools

Working software

over comprehensive documentation

Customer collaboration

over contract negotiation

Responding to change

over following a plan

On agile teams, people are less tied to one role and contribute across roles.

WHEN THE BOOK RECOMMENDS IT

- Unpredictable or changing requirements
- Responsible, collaborative developers
- Involved customers who understand and contribute to the process
- Popular methods: Scrum, eXtreme Programming, feature-driven development
- Agile keeps the SDLC's good practices. It refuses to let them block quick reactions to change

The three-schema architecture

ANSI/SPARC, 1978

- **External schema:** the users' views. A student sees her schedule, an advisor sees every advisee, the registrar sees every roster. Plus the enterprise data model
- **Conceptual schema:** all the external views combined into one coherent definition of the enterprise's data. The data architect's view
- **Internal schema:** really two. The logical schema (relational tables) and the physical schema (how this DBMS stores them)
- **Why three levels:** each group sees the database its own way, and a change at one level does not force changes at the others. Data independence with names

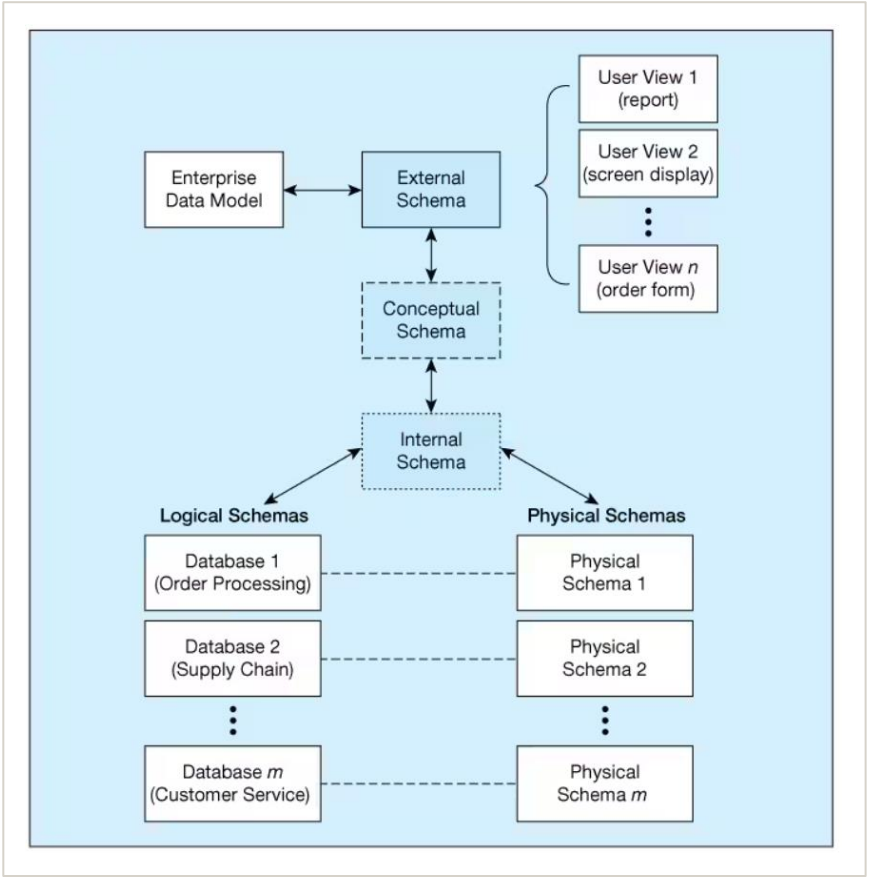


Figure 1-10. Three-schema architecture (Hoffer et al., 2026)

Who is on a database development team?

A project has a beginning, an end, and a project leader

ROLE	WHAT THEY DO
Business analysts	Analyze the business situation with management and users, write specifications
Systems analysts	Business analysis plus computer system requirements
Database analysts and data modelers	Requirements and design for the database component
Users	Assess their information needs and check that the system meets them
Programmers	Write the programs that maintain and access the data
Database architects	Set data standards across business units: location, currency, quality
Data administrators	Consistency and integrity across databases, consulting and training
Project managers	Team composition, analysis, design, implementation, support
Other technical experts	Networking, operating systems, testing, data warehousing, documentation

Evolution of database systems

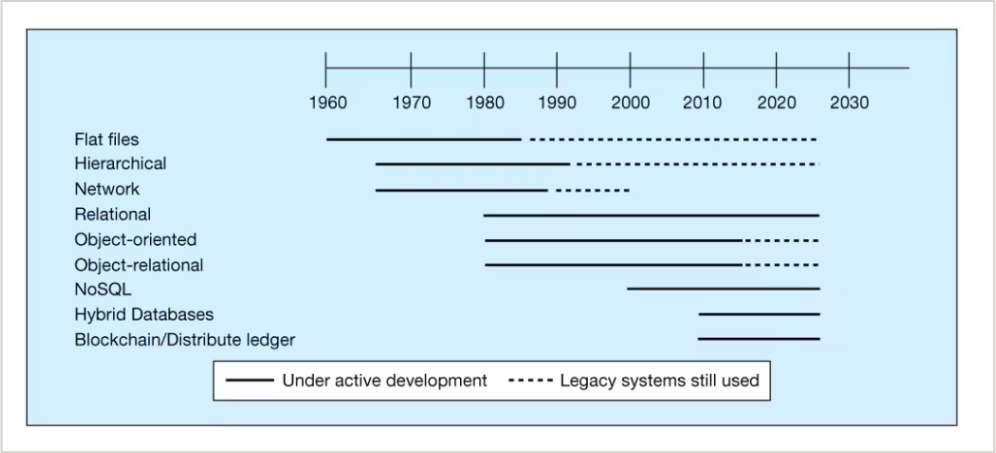


Figure 1-11(a). Evolution of database technologies (Hoffer et al., 2026)

- **1960s:** first DBMSs for projects like Apollo. Proof of concept
- **1970s:** hierarchical and network systems, the first generation. Still limited data independence
- **1980s:** the relational model (Codd, 1970) goes commercial: tables plus SQL. Second generation
- **1990s:** client/server, data warehousing, the Internet, multimedia. Object-oriented DBMSs (third generation) never took off
- **2000 and beyond:** relational still dominant. NoSQL (MongoDB, Cassandra), Hadoop and Spark, in-memory, cloud, hybrid SQL/NoSQL, blockchain

The organizing idea behind each technology

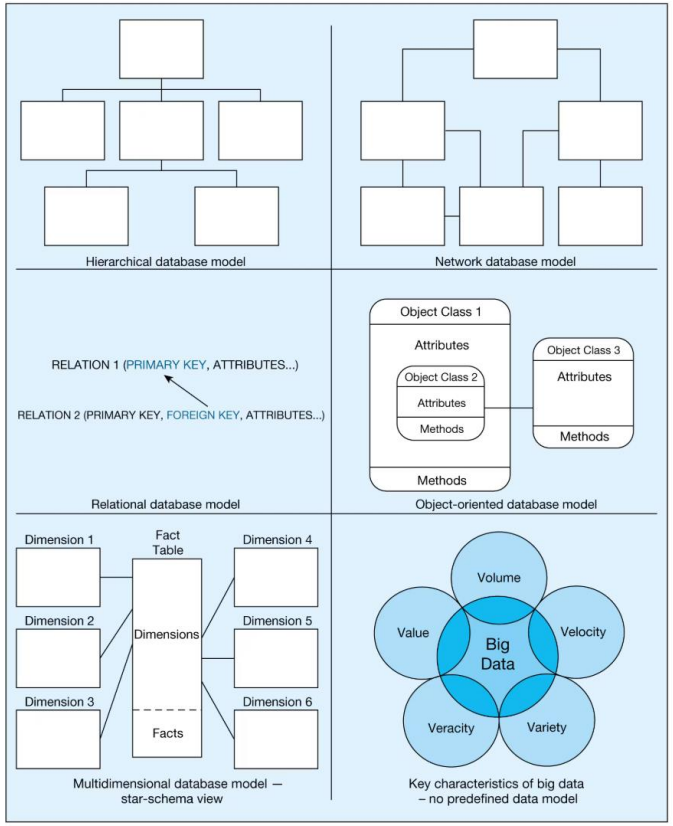


Figure 1-11(b). Database technologies (Hoffer et al., 2026)

- **Hierarchical:** a tree. Each child has one parent
- **Network:** a web. A record links to any number of others
- **Relational:** tables joined by primary key to foreign key. Our focus
- **Object-oriented:** classes bundling attributes with methods (Chapter 13)
- **Multidimensional:** a fact table surrounded by dimensions, the star schema of data warehouses (Chapter 10)
- **Big data:** no single model. The chapter names three Vs (volume, variety, velocity). The figure adds veracity and value

Personal, multi-tier, and enterprise databases

- **Two ways in:** ad hoc queries through the DBMS (needs SQL) or application programs (interface plus business logic). The client runs the interface, the database server runs the DBMS
- **Personal:** one user, megabytes, a laptop or a phone. Risk: data one person cares about soon interest others, and personal databases cannot be shared easily
- **Multi-tiered client/server:** a department or division. Client tier (browser or app), application/web tier (business logic), enterprise tier (DBMS). Blackboard works this way
- **Enterprise:** the whole organization or many departments. Often several per company

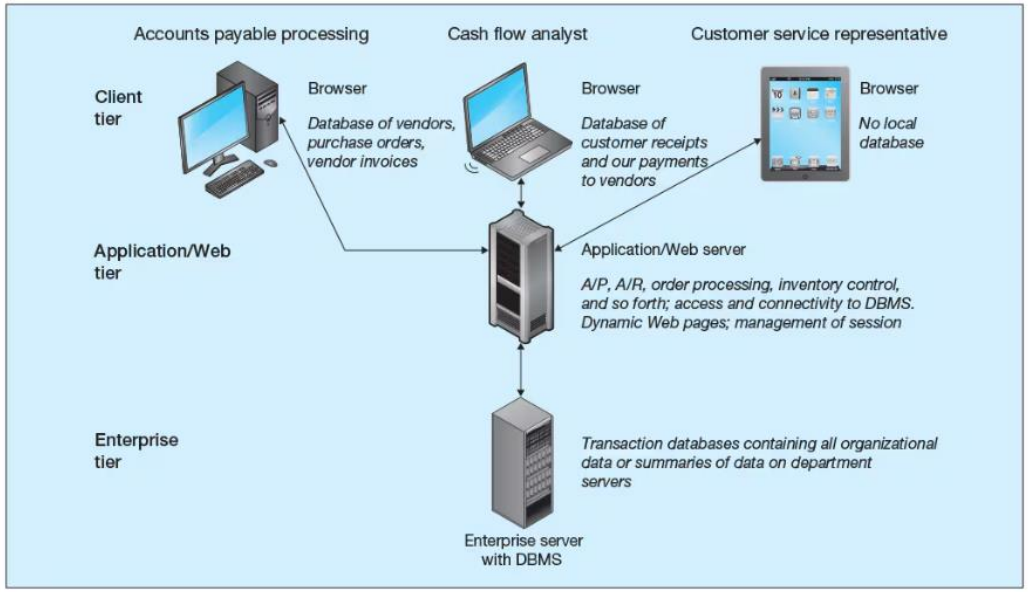
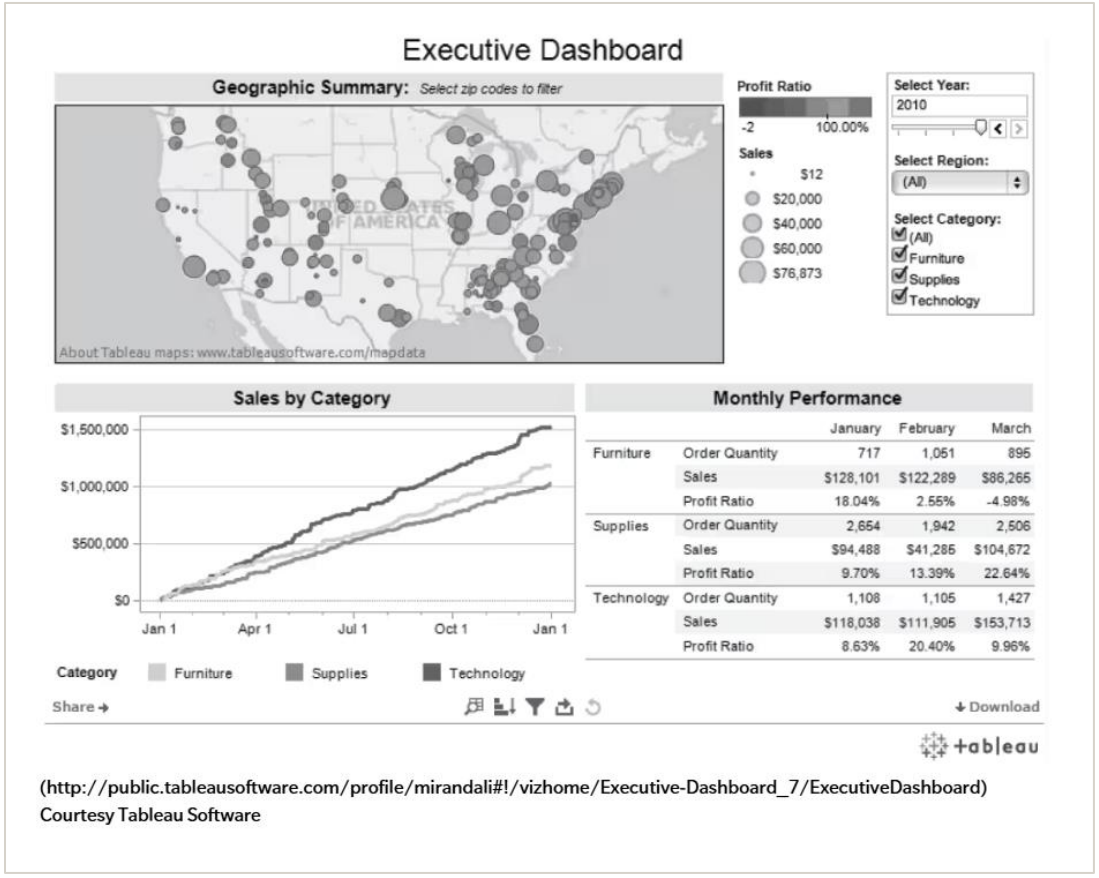


Figure 1-12. Multi-tiered client/server database architecture (Hoffer et al., 2026)

Three enterprise-level developments

- **Enterprise systems:** ERP, CRM, supply chain, HR, payroll. They capture transactions: registering for this course, buying a candy bar, Walmart paying 1.5 million hourly employees
- **Data warehouses:** collect history from many operational databases so users can find patterns and answer strategic questions. Dashboards like this one sit on top
- **Data lakes:** collect everything, no predefined schema (schema on read), cheap hardware, good for discovering new connections
- **Internet effects:** B2C on the public Internet, B2B on an extranet, employees on an intranet. New security questions, more so in the cloud



(http://public.tableausoftware.com/profile/mirandali#!/vizhome/Executive-Dashboard_7/ExecutiveDashboard)
Courtesy Tableau Software

Figure 1-13. An executive dashboard (Hoffer et al., 2026, courtesy Tableau Software)

Summary of database applications

Table 1-5

TYPE OF DATABASE OR APPLICATION	TYPICAL NUMBER OF USERS	TYPICAL SIZE
Personal	1	Megabytes
Multi-tiered client/server	2 to 1,000	Gigabytes
Enterprise resource planning	More than 100	Gigabytes to terabytes
Data warehousing	More than 100	Terabytes to petabytes
Data lake	More than 100	Terabytes to petabytes

- The book groups applications into three categories (personal, multi-tier, enterprise). The table splits enterprise into its three developments
- Same size ladder as the definition slide: megabytes for one person, petabytes for a warehouse or lake

4

PART 4

A database project at Pine Valley Furniture

Matteo needs to analyze home office sales. Helen builds him a database, from interview to working query.

The request, and four early decisions

- **Matteo Lopez,** product manager for home office furniture, wants direct access to sales data for ad hoc analysis
- **Helen Martin,** systems analyst, five years at PVFC, skilled in data modeling and several relational DBMSs
- **1 Stand-alone database,** so unpredictable analysis does not slow the operational systems
- **2 Extract from existing databases,** which PVFC already has
- **3 Prototyping within the life cycle,** because the data already exist and Matteo wants to analyze, not create
- **4 Microsoft Access,** PVFC's standard for personal databases

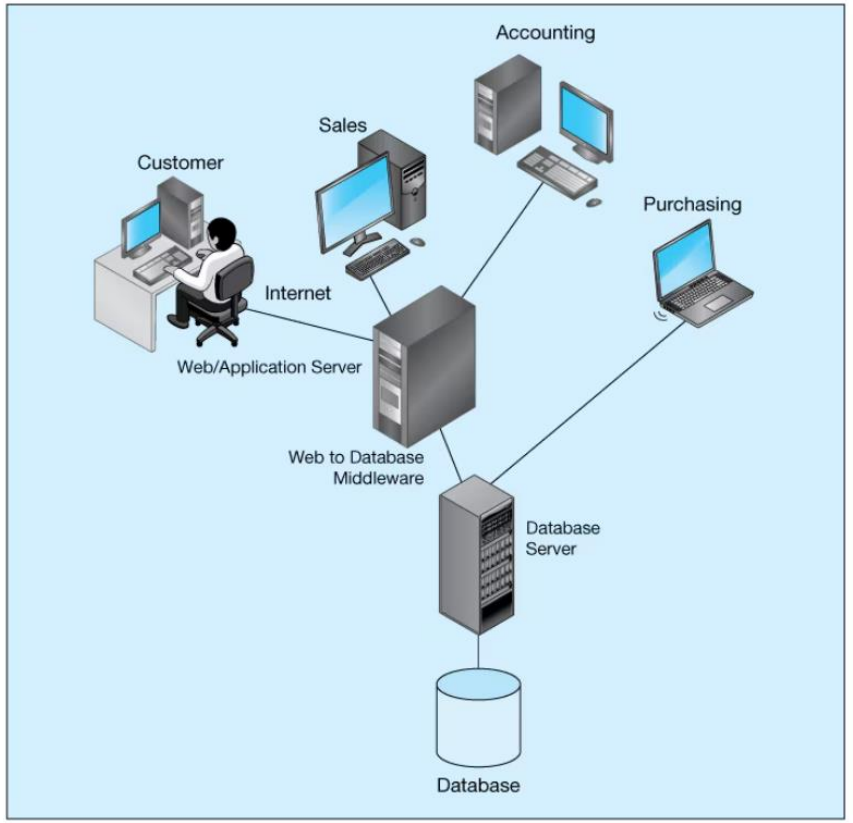


Figure 1-14. Computer system for Pine Valley Furniture Company (Hoffer et al., 2026)

Project planning: interview, then a first data model

- **The interview:** Helen listens more than she talks, uses marketing terms, and keeps Matteo on business needs instead of screens and reports
- **Two quick analyses:** which existing databases hold the entities he named, and which attributes matter. His objectives drive it: sales goals per finish, prior-year orders for the growth target, all products for comparison
- **The rules she reads back:** each CUSTOMER places any number of ORDERs, each ORDER is placed by exactly one CUSTOMER. Each ORDER contains many ORDER LINEs, each ORDER LINE is for exactly one PRODUCT. Each ORDER is billed on one INVOICE
- **Matteo adds:** customer years, shipments per order, prior and current year goals

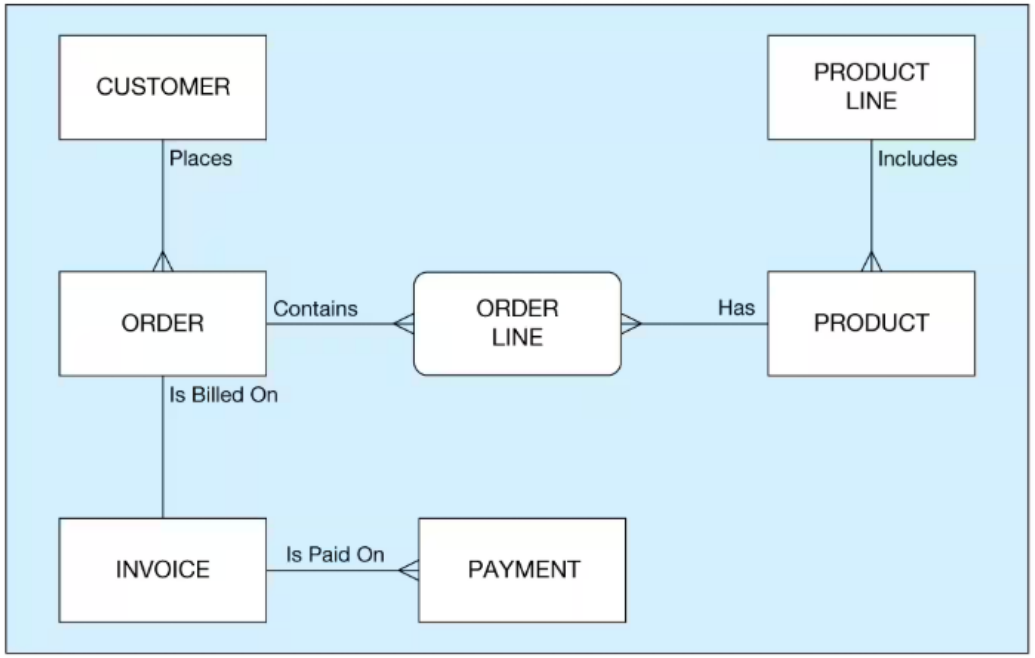


Figure 1-15. Preliminary data model for the Home Office marketing support system (Hoffer et al., 2026)

The project data model

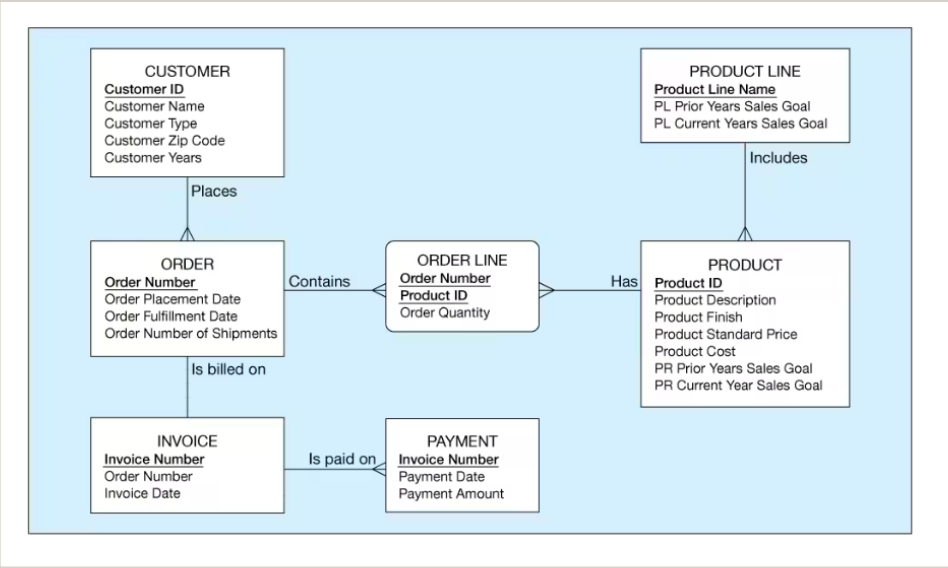


Figure 1-16. Project data model for the Home Office marketing support system (Hoffer et al., 2026)

- Same shapes as Figure 1-15, now with every attribute listed and each identifier underlined: Customer ID, Order Number, Product ID, Product Line Name, Invoice Number
- ORDER LINE has a two-part identifier, Order Number plus Product ID: one line is one product on one order
- The model records how the business works and what constraints it lives with. Next step: turn each entity into a table

From entities to tables

- **Naming:** tables end in _T, no spaces (ProductDescription), every table has a primary key with a unique value per row
- **Trace it:** which customer placed order 1001? Order_T says CustomerID 1. Customer_T says Contemporary Casuals, Gainesville, Florida
- **Try it:** which products are on order 1004? OrderLine_T has two rows: ProductID 6 (quantity 2) and ProductID 8 (quantity 2). Product_T: an 8-Drawer Desk in White Ash at $750 and a Computer Desk in Walnut at $250
- **Order value:** 2 x 750 + 2 x 250 = $2,000. In Chapter 5 you will write one SQL statement for this
- **Note:** the chapter text says order 1003 belongs to customer 1. The figure shows customer 15. Trust the figure

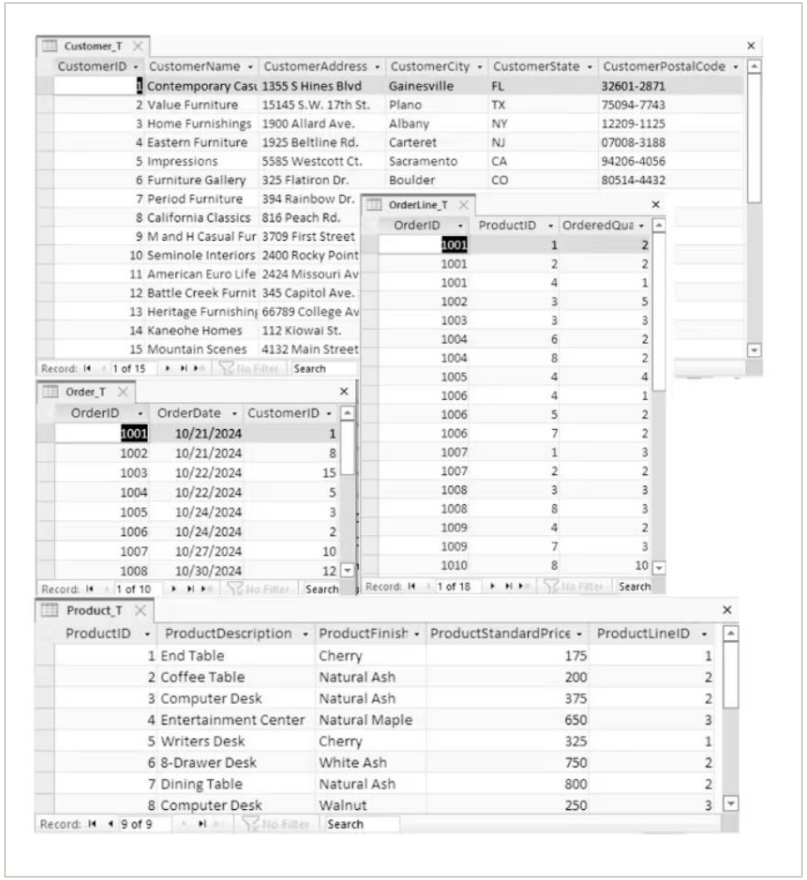


Figure 1-17. Four relations (Hoffer et al., 2026)

Creating a table in SQL

```
CREATE TABLE Product_T
(ProductID              NUMBER(11,0) NOT NULL PRIMARY KEY
ProductDescription      VARCHAR (50),
ProductFinish           VARCHAR (20),
ProductStandardPrice    DECIMAL(6,2),
ProductCost             DECIMAL,
ProductPriorYearGoal    DECIMAL,
ProductCurrentYearGoal  DECIMAL,
ProductLineID           VARCHAR (40),
FOREIGN KEY             (ProductLineID)  REFERENCES ProductLine_T (ProductLineID));
```

Figure 1-19. SQL definition of the Product_T table (Hoffer et al., 2026)

- **VARCHAR(50):** variable-length text up to 50 characters. NUMBER(11,0): a whole number up to 11 digits. DECIMAL(6,2): up to 6 digits with 2 after the point, such as 1234.56
- **NOT NULL:** every row must have a value. PRIMARY KEY: the unique identifier, never empty, never repeated
- **FOREIGN KEY (ProductLineID) REFERENCES ProductLine_T:** a product's line must already exist in ProductLine_T. That is the relational integrity constraint from the advantages list, and it means ProductLine_T gets created first

Physical design: which columns get an index?

- An index is like the index at the back of a textbook: jump to the right rows instead of scanning every one
- The DBMS indexes every primary key, because the values are unique
- **Helen's rule of thumb:** index any attribute with more than 10 different values that Matteo will use to segment the data
- **Product Finish?** No. PVFC uses only six finishes, so an index barely helps
- **Order Placement Date?** Yes. Many values, and Matteo analyzes sales by period. The book calls it a secondary key, because values can repeat
- Because the system is for decision support, nobody can predict every query. Helen decides from experience, not from a fixed list of transactions

Using the database: the first prototype query

Home Office products, sales to date against the current-year goal

```
SELECT Product.ProductID, Product.ProductDescription, Product.PRCurrentYearSalesGoal,
            (OrderQuantity * ProductPrice) AS SalesToDate
FROM Order.OrderLine, Product.ProductLine
WHERE Order.OrderNumber = OrderLine.OrderNumber
AND Product.ProductID = OrderedProduct.ProductID
AND Product.ProductID = ProductLine.ProductID
AND Product.ProductLineName = 'Home Office';
```

Figure 1-20. SQL query (Hoffer et al., 2026)

Home Office Sales to Date : Select Query

Product ID	Product Description	PR Current Year Sales Goal	Sales to Date
3	Computer Desk	$23,500.00	5625
10	96" Bookcase	$22,500.00	4400
5	Writer's Desk	$26,500.00	650
3	Computer Desk	$23,500.00	3750
7	48" Bookcase	$17,000.00	2250
5	Writer's Desk	$26,500.00	3900

Figure 1-21. Query output (Hoffer et al., 2026)

- **SELECT** the columns to show, plus a calculation: OrderQuantity times ProductPrice, labeled SalesToDate
- **FROM** the tables the data come from
- **WHERE** links the tables by matching keys, then keeps only the Home Office line
- Computer Desk appears twice in the output because each row is one order line. Chapter 5 adds GROUP BY to total them
- The book writes the query in a shorthand that will not run as written. Read the three clauses, and see the runnable JOIN version in Chapter 5

What PVFC's databases still cannot answer

- What is the pattern of furniture sales this year compared with the same period last year?
- Who are our ten largest customers, and what are their buying patterns?
- Why can we not get one consolidated view of a customer who orders through several sales channels?
- Operational databases run the business well. Decision support needs a separate database of historical, summarized data: a data warehouse or a data mart, with OLAP tools on top
- Chapters 9 through 11 return to PVFC's warehouse plans, and maybe a data lake

Chapter 1 in one slide

The lists you should be able to write from memory

5 disadvantages of file processing

10 advantages of the database approach

5 costs and risks

9 components of the database environment

5 SDLC phases, with the database activity in each

3 schemas: external, conceptual, internal (logical + physical)

4 agile values

3 categories of database applications

4 drivers of database evolution

Plus the definitions: database, data, information, metadata, entity, attribute, relationship, DBMS, repository, user view, constraint, data independence, prototyping, data warehouse, data lake.

Before next Wednesday

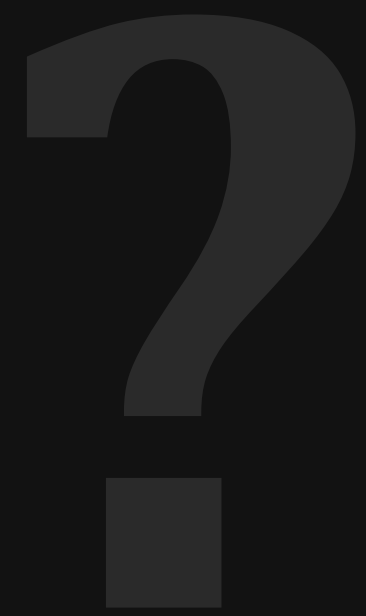
- **Chapter 1 tasks on Blackboard:** quiz (5 points), matching exercise (5 points), written assignment (8 points). Due Wednesday, September 2, before class
- **Graduate students:** add the graduate extension to the written assignment, with at least one scholarly or practitioner source
- **Read Chapter 2,** Modeling Data in the Organization. Bring Draw.io questions
- **Use the study guide:** every figure from tonight with plain-language explanations, worked examples, and practice questions
- **Office hours:** Wednesdays 4:00 to 5:00 PM, Woolsey Hall 308L. Email with MIS 600 in the subject line
- **Reminder:** no late work, and the drop policy is the safety net. The last day to drop with a full refund is August 28

Questions

nawaf.aljohani@wichita.edu

Woolsey Hall 308L · Wednesdays 4:00 to 5:00 PM

Ask My Instructor forum on Blackboard



Beck, K., et al. (2001). Manifesto for Agile Software Development. https://agilemanifesto.org

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387.

Hoffer, J. A., Topi, H., Venkataraman, R., & Bala, H. (2026). Modern database management (14th ed.). Pearson.